

01 · Descripción general del sistema

Documento	01-system-overview.md
Versión analizada	0.5.1 (commit f840055)
Fecha del análisis	2026-08-28
Fuente Markdown	docs/system-documentation/01-system-overview.md
Generado por	scripts/docs/build-pdf.mjs

Contenido

ChofyAI Studio · 01 · Descripción general del sistema

v0.5.1 (commit f840055) · 2026-08-28

1. Qué es ChofyAI Studio
2. Qué problema resuelve
3. A quién está dirigido
4. Herramientas integradas
5. Casos de uso principales
6. Actores del sistema
7. Flujo general de funcionamiento
8. Entradas y salidas
 - Entradas
 - Salidas
9. Componentes más importantes
10. Tecnologías y dependencias
11. Límites del sistema
12. Integraciones externas
13. Estado observado del repositorio
14. El sistema explicado para una persona no técnica
15. Por dónde seguir

Estado: completo · Última revisión: 2026-08-27 · Versión analizada: 0.5.1 (commit f840055)

1. Qué es ChofyAI Studio

ChofyAI Studio es una **aplicación de escritorio** que instala, arranca, vigila y detiene herramientas de inteligencia artificial que se ejecutan **en el propio computador del usuario**, sin enviar datos a ningún servicio remoto durante la inferencia.

No es un modelo de IA ni una herramienta creativa por sí misma: es el **orquestador** que se encarga del trabajo aburrido y frágil que hay alrededor de esas herramientas — descargar el código, crear entornos de Python, compilar binarios, bajar modelos, elegir puertos, lanzar procesos, guardar registros y volver a levantar todo tras un cierre inesperado.

Evidencia en el repositorio:

- El backend registra 35 comandos en `src-tauri/src/lib.rs` , todos orientados a instalar, ejecutar, inspeccionar o reubicar herramientas.
- Cada herramienta se describe en un manifiesto YAML de `apps/` y se instala con un script de `scripts/mac/` o `scripts/win/` .
- La interfaz completa vive en `src/App.tsx` .

2. Qué problema resuelve

Instalar herramientas de IA locales en un Mac es, en la práctica, una cadena de pasos manuales que falla con facilidad. El repositorio contiene la evidencia de esos fallos: el postmortem `../POSTMORTEM-2026-05-17.md` documenta diez incidentes reales, y varios de ellos están hoy mitigados dentro del código.

Problema real	Dónde se resolvió
Cada herramienta se instala distinto (venv, cmake, conda, uv)	<code>scripts/mac/common.sh</code> unifica detección de Python, creación de entorno e instalación de paquetes
Una instalación "termina bien" pero queda incompleta	Post-validación de <code>installed_if</code> al final de <code>run_install_script</code> en <code>src-tauri/src/system.rs</code>
Un proceso zombi ocupa el puerto y la herramienta "no abre"	Pre-flight de puerto en <code>start_tool</code> , más la detección de huérfanos de <code>list_orphan_ports</code>
El disco externo se desmonta y desaparecen todas las herramientas	<code>resolve_effective_home</code> intenta montar el <code>.sparsebundle</code> y, si no puede, cae a <code>~/ChofyAISTudio</code>
No se sabe qué está pasando durante una instalación larga	Eventos <code>install-progress</code> y el parser de fases <code>parseInstallLine</code> de <code>src/utils.ts</code>
Los modelos pesados no caben en el disco principal	<code>relocate_module</code> y los overrides <code>models_dir</code> / <code>outputs_dir</code> / <code>cache_dir</code>

3. A quién está dirigido

Perfil	Qué obtiene
Creador de contenido en Mac	Instala y usa voz, transcripción, imagen, música y video sin tocar la terminal
Usuario preocupado por la privacidad	Toda la inferencia ocurre en <code>127.0.0.1</code> ; no hay cuenta ni telemetría en el código
Desarrollador que evalúa herramientas	Instalación reproducible y desechable, con logs y control de procesos
Usuario con disco externo	Soporte explícito de volúmenes externos, sparsebundle APFS y reubicación por herramienta

No está dirigido a despliegues multiusuario ni a servidores: no hay autenticación, ni roles, ni API remota. Véase [11 · Seguridad](#).

4. Herramientas integradas

Datos tomados de los cinco manifiestos de `apps/` .

Herramienta	Categoría	Runtime	Puerto	Plataformas declaradas
Qwen3-TTS	<code>voice</code>	<code>python</code> (MLX)	7860	<code>mac-arm64</code>
whisper.cpp	<code>asr</code>	<code>binary</code>	8178	<code>mac-arm64</code> , <code>win-x64</code> , <code>linux-x64</code>
FaceFusion	<code>video</code>	<code>python</code>	7862	<code>mac-arm64</code> , <code>win-x64</code> , <code>linux-x64</code>
AceForge	<code>music</code>	<code>python</code>	7857	<code>mac-arm64</code> , <code>win-x64</code> , <code>linux-x64</code>
ComfyUI	<code>image</code>	<code>python</code>	8188	<code>mac-arm64</code> , <code>win-x64</code> , <code>linux-x64</code>

Aviso de precisión: que un manifiesto declare `win-x64` o `linux-x64` **no** significa que esa plataforma esté validada. Los scripts de Linux referenciados (`scripts/linux/install-*.sh`) **no existen** en el repositorio, y el propio manifiesto los marca con un comentario `# TODO: pendiente` . Los de Windows existen en `scripts/win/` pero el `README.md` los describe como experimentales sin validación de extremo a extremo. Detalle en [15 · Riesgos y deuda técnica](#).

5. Casos de uso principales

- Primera puesta en marcha:** el asistente `Onboarding` de `src/App.tsx` propone una ubicación de trabajo, comprueba el sistema de archivos y ofrece instalar `whisper.cpp` como primera herramienta.

2. **Instalar una herramienta:** tarjeta → botón *Instalar* → comprobación previa de espacio

(`PreInstallCheck`) → ejecución del script con progreso en vivo.

v0.5.1 (commit f840055) · 2026-08-28

3. **Instalar varias en lote:** cola secuencial (`addAllPendingToQueue` + `runQueue`), una herramienta a la vez para no saturar disco y red.
4. **Usar una herramienta:** *Iniciar* → health check → *Ver UI*, que embebe la interfaz web de la herramienta en un `iframe` dentro de la propia aplicación.
5. **Gestionar modelos:** listar lo que hay en disco, descargar los repositorios de Hugging Face declarados en el manifiesto y borrar los que sobran (`ModelsPanel`).
6. **Mover una herramienta de disco:** `relocate_module` traslada el directorio y registra un override persistente en `settings.json`.
7. **Recuperarse de un cierre forzado:** al arrancar se restauran los PID vivos y se detectan procesos huérfanos que ocupan puertos conocidos.
8. **Encadenar herramientas:** los workflows YAML de `workflows/` describen pipelines HTTP; el constructor visual permite crear nuevos.
9. **Diagnosticar el entorno:** `run_doctor` ejecuta `scripts/mac/doctor.sh` y muestra la salida en la interfaz.

6. Actores del sistema

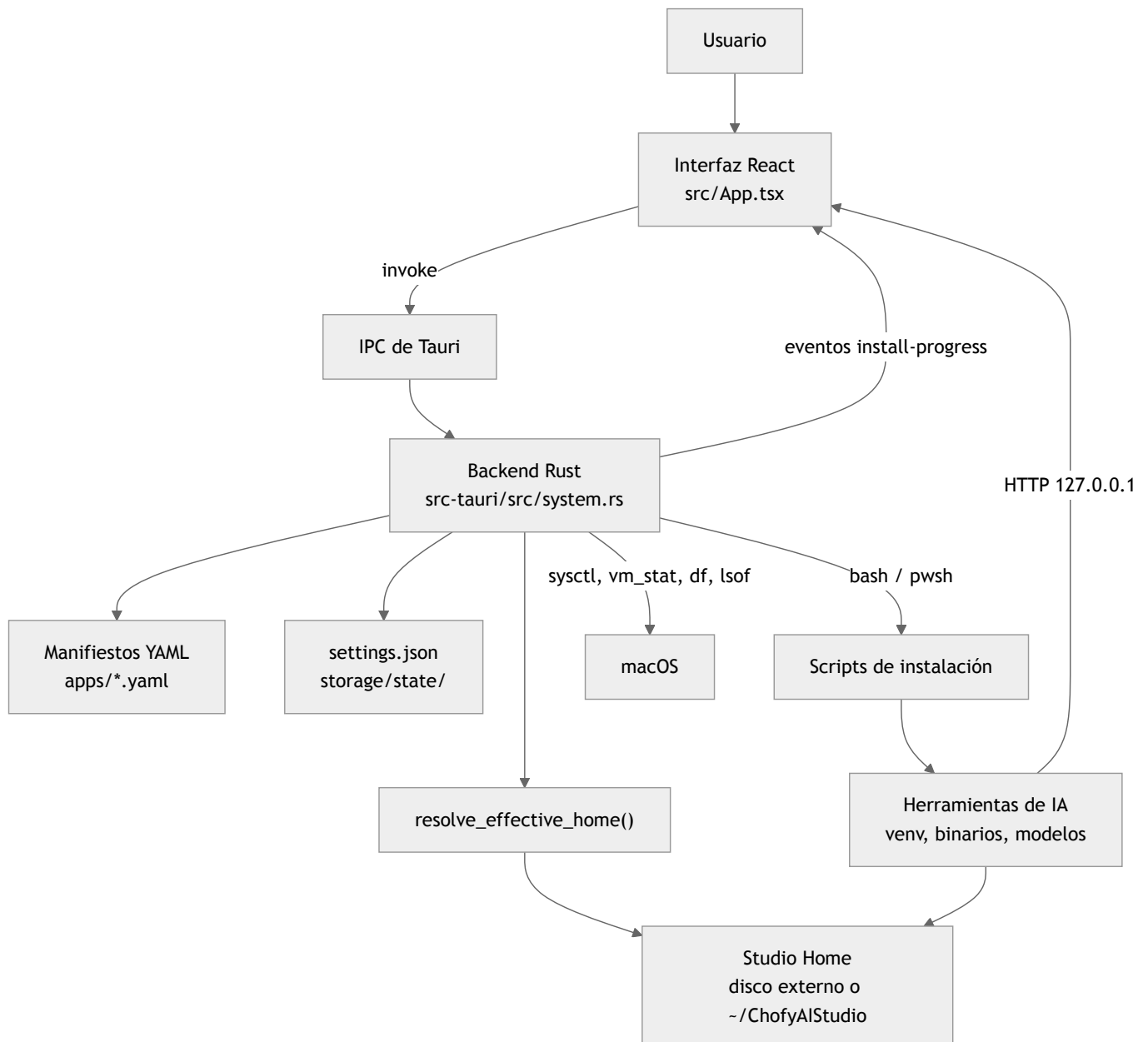
Sólo existe un tipo de usuario humano. El resto son actores técnicos.

Actor	Rol
Usuario local	Único operador; tiene control total, sin roles ni permisos internos
Aplicación (WebView React)	Interfaz, estado de sesión, temporizadores y llamadas IPC
Backend Rust	Ejecuta acciones privilegiadas: procesos, disco, red local, utilidades del sistema
Scripts de shell	Instalan y actualizan cada herramienta
Herramientas de IA	Procesos independientes que sirven su propia interfaz HTTP en <code>127.0.0.1</code>
Servicios externos	GitHub, Hugging Face, PyPI, Homebrew: sólo durante instalación y descarga

7. Flujo general de funcionamiento

ChofyAI Studio · 01 · Descripción general del sistema

v0.5.1 (commit f840055) · 2026-08-28



Lectura del diagrama: la interfaz nunca toca el disco ni lanza procesos por su cuenta; todo pasa por el backend Rust, que antes de cualquier operación resuelve dónde vive el *Studio Home*. Los scripts de instalación reciben esa ruta por variable de entorno, y las herramientas instaladas terminan sirviendo su propia interfaz web, que la aplicación embebe. El único canal de vuelta hacia la interfaz son los valores de retorno de los comandos y los eventos de progreso.

8. Entradas y salidas

Entradas

- Acciones del usuario en la interfaz (instalar, arrancar, mover, borrar).
- Manifiestos YAML de `apps/`, catálogo `marketplace/registry.yaml` y workflows.

- `storage/state/settings.json` con la configuración persistida.

- Estado del sistema operativo: CPU, memoria, disco, volúmenes montados, puertos ocupados
- Contenido descargado de internet durante la instalación (código y modelos).

Salidas

- Árbol de directorios bajo el *Studio Home*: `tools/` , `models/` , `outputs/` , `cache/` , `logs/` , y `modules/` si se reubican herramientas.
- Registros: `<tool>-install.log` , `<tool>-run.log` , `<tool>-model-download.log` y `crash.log` .
- Estado persistido: `settings.json` y `processes.json` .
- Procesos en ejecución escuchando en `127.0.0.1` .
- Notificaciones nativas de macOS vía `osascript` .

9. Componentes más importantes

Componente	Ubicación	Responsabilidad
Resolutor de rutas	<code>resolve_effective_home</code> en <code>system.rs</code>	Decide dónde se instala y ejecuta todo; incluye auto-montaje y fallback
Cargador de manifiestos	<code>collect_manifests</code> , <code>RawManifest</code>	Convierte YAML en el catálogo de herramientas
Ejecutor de instalación	<code>run_install_script</code>	Lanza el script, transmite progreso y valida el resultado
Registro de procesos	<code>ProcessRegistry</code> + <code>processes.json</code>	Asocia herramienta ↔ PID y sobrevive a reinicios
Supervisión	<code>health_check_tool</code> , <code>list_orphan_ports</code>	Sabe qué está vivo y qué quedó suelto
Interfaz	<code>src/App.tsx</code>	Todo el estado de sesión, temporizadores y paneles
Parser de progreso	<code>parseInstallLine</code> en <code>src/utils.ts</code>	Traduce la salida cruda de git/cmake/pip a fases legibles
Generador de documentación	<code>scripts/docs/build-pdf.mjs</code>	Deriva los PDF de esta documentación

10. Tecnologías y dependencias

Capa	Tecnología	Versión declarada
Shell de escritorio	Tauri 2	<code>tauri = "2" en src-tauri/Cargo.toml</code>
Backend	Rust edición 2021	Dependencias: <code>serde</code> , <code>serde_json</code> , <code>serde_yaml</code> , <code>thiserror</code> , <code>walkdir</code>
Interfaz	React 18 + TypeScript + Vite	<code>react ^18.3.1</code> , <code>typescript ^7.0.2</code> , <code>vite ^8.1.4</code>
Pruebas	Vitest y <code>cargo test</code>	<code>vitest ^4.1.10</code>
Gestor de paquetes	pnpm fijado por Corepack	<code>pnpm@10.29.3</code>
Scripts	Bash y PowerShell	<code>scripts/mac/</code> , <code>scripts/win/</code>
Manifiestos	YAML	<code>apps/</code> , <code>workflows/</code> , <code>marketplace/</code>

Llama la atención lo corta que es la lista de dependencias de Rust: las estadísticas del sistema se leen invocando utilidades de macOS en lugar de añadir una biblioteca. Es una decisión deliberada, con la contrapartida de atar el código a macOS (véase [15 · Riesgos](#)).

11. Límites del sistema

Lo que ChofyAI Studio **no** hace, verificado en el código:

- No expone ningún servidor propio ni API remota.
- No tiene autenticación, cuentas, roles ni multiusuario.
- No ejecuta inferencia: eso lo hacen las herramientas que instala.
- No tiene base de datos; persiste en JSON, YAML y el árbol de ficheros ([07 · Base de datos y persistencia](#)).
- No hay telemetría ni analítica en el código analizado.
- No actualiza la aplicación automáticamente: `UpdateChecker` sólo consulta el último release en GitHub y avisa.
- No hay monitorización, alertas ni respaldo automatizado ([13 · Despliegue y operación](#)).

12. Integraciones externas

ChofyAI Studio · 01 · Descripción general del sistema

v0.5.1 (commit f840055) · 2026-08-28

Servicio	Cuándo se usa	Qué se envía
GitHub (git clone)	Instalación de cada herramienta	Nada del usuario; se descarga la rama por defecto
API de GitHub Releases	<code>UpdateChecker</code> al abrir la aplicación	Sólo la petición HTTP; no se envían datos locales
Hugging Face	Descarga de modelos declarados	Identificador del repositorio solicitado
PyPI (pip / uv)	Instalación de dependencias Python	Nombres de paquetes
Homebrew	Instalación de <code>ffmpeg</code> en dos scripts	Nada del usuario
jsDelivr	Sólo al generar los PDF de esta documentación	Nada del usuario

Detalle completo, con URLs y formatos, en [09 · APIs e integraciones](#).

13. Estado observado del repositorio

- Versión declarada en `package.json` y `src-tauri/Cargo.toml`: **0.5.1**; `src-tauri/tauri.conf.json` y la constante `APP_VERSION` de `src/App.tsx` siguen en **0.5.0**. Es una discrepancia real, registrada como riesgo.
- Cinco herramientas integradas, todas con script de macOS; cuatro con script de Windows; ninguna con script de Linux pese a estar referenciado.
- Pruebas automatizadas: dos archivos en el frontend (`utils.test.ts`, `i18n.test.ts`) y cuatro pruebas en Rust dentro de `system.rs`. La interfaz no tiene pruebas.
- Integración continua con cinco trabajos en `ci.yml` y escaneo de seguridad semanal.
- Documentación previa abundante en `../`, ahora complementada por este set.

14. El sistema explicado para una persona no técnica

Imagine que quiere usar en su computador varios programas de inteligencia artificial: uno que convierte texto en voz, otro que transcribe audio, otro que genera imágenes, otro que hace música y otro que trabaja con caras en video. Todos existen y son gratuitos, pero cada uno se instala de una manera distinta, tarda mucho, ocupa muchos gigabytes y falla por motivos difíciles de entender.

ChofyAI Studio es una aplicación con botones que hace todo eso por usted. Muestra una tarjeta por cada programa, con un botón de *Instalar*. Al pulsarlo, la aplicación descarga lo necesario y le va contando en qué paso va: descargando, compilando, instalando. Cuando termina, el botón cambia a *Iniciar*, y al pulsarlo el programa se abre dentro de la misma ventana, como si fuera una pestaña.

Tres detalles que la hacen distinta de simplemente seguir un tutorial:

1. **Todo ocurre en su computador.** Ni su voz, ni sus fotos, ni sus textos salen del equipo cuando usa las herramientas.

2. **Sabe dónde guardar las cosas.** Estos programas ocupan mucho espacio, así que puede elegir un disco externo. Si un día ese disco no está conectado, la aplicación se da cuenta, intenta montarlo sola y si no puede, sigue funcionando con una carpeta del disco principal en vez de romperse.
3. **Limpia lo que queda mal.** Si algo se quedó a medias o un programa siguió corriendo tras cerrar la aplicación, lo detecta y le ofrece adoptarlo o cerrarlo.

Lo que la aplicación **no** hace: no crea la voz ni la imagen por sí misma —eso lo hacen los programas que instala—, no guarda nada suyo en internet y no tiene usuarios ni contraseñas: quien tenga acceso al computador tiene acceso a la aplicación.

15. Por dónde seguir

Si busca...	Vaya a
Instalarlo y ejecutarlo	02 · Instalación y ejecución
Entender la arquitectura	03 · Arquitectura
Localizar un archivo o función	04 · Mapa del código
La firma exacta de algo	05 · Referencia técnica
Cómo funciona por dentro	06 · Explicación profunda
Un resumen para decidir	17 · Resumen ejecutivo
Incorporarse al proyecto	18 · Guía para nuevos desarrolladores